

Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій
Кафедра радіоелектронних і комп'ютерних систем

Звіт
Про виконання лабораторної роботи №1
з курсу
“Машинне навчання”
«Множинна лінійна регресія методом градієнтного спуску»

Виконав:
ст. гр. ФЕІ-44
Політовський Назар
Перевірив:
доц. Сінькевич О. О.

Мета роботи: Реалізувати алгоритм множинної лінійної регресії з нуля за допомогою методу градієнтного спуску. Провести повний аналіз якості моделі, включаючи перевірку всіх шести класичних припущень МНК.

Хід роботи:

1. Написання коду програми:

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

import statsmodels.api as sm
from statsmodels.stats.outliers_influence import variance_inflation_factor
from statsmodels.stats.stattools import durbin_watson
from scipy.stats import shapiro, probplot

# =====
# 2. Завантаження та EDA
# =====
# Завантажуємо датасет
df = pd.read_csv("Student_Performance.csv")

# ◇ Перетворення категоріальної змінної у числову одразу після завантаження
# Це потрібно для подальшого використання у регресії
if "Extracurricular Activities" in df.columns:
    df['Extracurricular Activities'] = (
        df['Extracurricular Activities'].map({'No': 0, 'Yes': 1}).astype(int)
    )

# Виводимо загальну інформацію про датасет (типи колонок, пропущені значення)
print("◇ Інформація про датасет:")
print(df.info())
print(df.describe())

# Перевірка на наявність пропущених значень
print("\n◇ Перевірка пропущених значень:")
print(df.isnull().sum())

# ◇ Візуалізація кореляційної матриці
plt.figure(figsize=(8, 6))
sns.heatmap(df.corr(), annot=True, cmap="coolwarm")
plt.title("Кореляція ознак")
plt.show()

# ◇ Перевірка правильності кодування категоріальних змінних
print("\n◇ Перевірка типів даних перед train/test:")
print(df.dtypes)

print("\n◇ Унікальні значення Extracurricular Activities:")
```

```

print(df["Extracurricular Activities"].unique())

# =====
# 3. Попередня обробка
# =====
# Вибираємо цільову змінну та ознаки
y = df["Performance Index"] # Змінна, яку будемо прогнозувати
X = df.drop("Performance Index", axis=1) # Всі інші колонки – ознаки

# ◊ Поділ на тренувальний та тестовий набори (80/20)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# ◊ Масштабування ознак (важливо для градієнтного спуску)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# ◊ Перевірка на NaN після масштабування
print("\n ◊ Перевірка NaN після масштабування:")
print("NaN у X_train:", np.isnan(X_train_scaled).sum())
print("NaN у X_test:", np.isnan(X_test_scaled).sum())

# ◊ Страхівка: замінюємо NaN та inf на 0
X_train_scaled = np.nan_to_num(X_train_scaled)
X_test_scaled = np.nan_to_num(X_test_scaled)

# ◊ Додаємо стовпець одиниць для  $\beta_0$  (вільного члена регресії)
X_train_final = np.c_[np.ones(X_train_scaled.shape[0]), X_train_scaled]
X_test_final = np.c_[np.ones(X_test_scaled.shape[0]), X_test_scaled]

# =====
# 4. Градієнтний спуск
# =====
# Функція для обчислення середньоквадратичної помилки
def compute_cost(X, y, b):
    predictions = X.dot(b) # Обчислюємо прогноз
    errors = predictions - y # Обчислюємо похибку
    mse = (1 / len(y)) * np.sum(errors ** 2) # MSE
    return mse

# Основна функція градієнтного спуску
def gradient_descent(X, y, b, learning_rate=0.01, epochs=1000):
    cost_history = [] # Історія помилок для візуалізації збіжності
    m = len(y) # Кількість спостережень
    for _ in range(epochs):
        predictions = X.dot(b) # Поточні прогнози
        errors = predictions - y # Розрахунок похибки
        gradient = (2 / m) * X.T.dot(errors) # Градієнт MSE
        b = b - learning_rate * gradient # Оновлюємо коефіцієнти
        cost_history.append(compute_cost(X, y, b)) # Зберігаємо поточну помилку
    return b, cost_history

# ◊ Ініціалізація коефіцієнтів нулями

```

```

b_init = np.zeros(X_train_final.shape[1])

# ◇ Навчання моделі методом градієнтного спуску
b_final, cost_history = gradient_descent(
    X_train_final, y_train.values, b_init, 0.01, 1000
)

# ◇ Візуалізація збіжності градієнтного спуску
plt.plot(range(len(cost_history)), cost_history)
plt.xlabel("Ітерації")
plt.ylabel("MSE")
plt.title("Збіжність градієнтного спуску")
plt.show()

# ◇ Виведення фінальних коефіцієнтів
print("◇ Фінальні коефіцієнти:")
print(b_final)

# =====
# 5. Оцінка моделі
# =====
# Прогнозування на тестовому наборі
y_pred = X_test_final.dot(b_final)

# Обчислення MSE та R² на тесті
mse_test = np.mean((y_test - y_pred) ** 2)
r2 = 1 - (
    np.sum((y_test - y_pred) ** 2)
    / np.sum((y_test - np.mean(y_test)) ** 2)
)

print(f"\nMSE (тест): {mse_test:.4f}")
print(f"R² (тест): {r2:.4f}")

# =====
# 6. Перевірка 6 припущень МНК
# =====

# 1. Лінійність та гомоскедастичність
residuals = y_test - y_pred # Залишки
plt.scatter(y_pred, residuals)
plt.axhline(0, color="red")
plt.xlabel("Прогнозовані значення")
plt.ylabel("Залишки")
plt.title("Залишки vs Прогнозовані значення")
plt.show()

# 2. Мультиколінеарність (VIF)
vif = pd.DataFrame()
vif["VIF"] = [
    variance_inflation_factor(X_train_scaled, i)
    for i in range(X_train_scaled.shape[1])
]
vif["feature"] = X.columns
print("\n◇ VIF показники:")

```

```

print(vif)

# 3. Нормальність залишків
probplot(residuals, dist="norm", plot=plt)
plt.title("Q-Q plot залишків")
plt.show()

shapiro_test = shapiro(residuals) # Shapiro-Wilk тест на нормальність
print("\n ◇ Shapiro-Wilk test:", shapiro_test)

# 4. Автокореляція залишків
dw = durbin_watson(residuals)
print("\n ◇ Durbin-Watson:", dw)

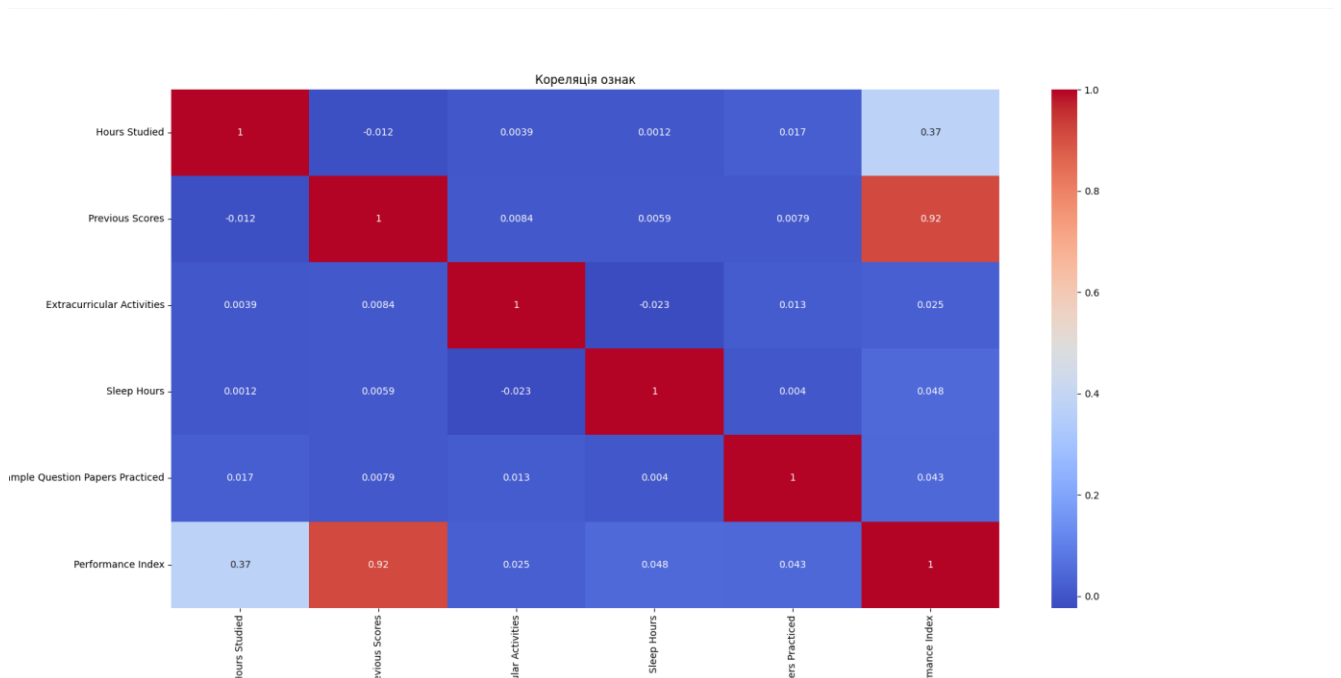
# 5. Викиди (Cook's Distance)
model = sm.OLS(y_train, X_train_final).fit() # OLS на тренувальних даних
influence = model.get_influence() # Вплив кожного спостереження
cooks_d = influence.cooks_distance[0]

# Візуалізація Cook's Distance
plt.stem(np.arange(len(cooks_d)), cooks_d, markerfmt=",")
plt.axhline(4 / len(y_train), color="red", linestyle="--") # Порогове значення
plt.title("Cook's Distance")
plt.show()

```

Результат виконання роботи:

Графік 1. Кореляція ознак:



```

♦ Інформація про датасет:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 6 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Hours Studied                        10000 non-null  int64
1   Previous Scores                     10000 non-null  int64
2   Extracurricular Activities          10000 non-null  int64
3   Sleep Hours                        10000 non-null  int64
4   Sample Question Papers Practiced    10000 non-null  int64
5   Performance Index                   10000 non-null  float64
dtypes: float64(1), int64(5)
memory usage: 468.9 KB
None

```

	Hours Studied	Previous Scores	Extracurricular Activities	Sleep Hours	Sample Question Papers Practiced	Performance Index
count	10000.000000	10000.000000	10000.000000	10000.000000	10000.000000	10000.000000
mean	4.992900	69.445700	0.494800	6.530600	4.583300	55.224800
std	2.589309	17.343152	0.499998	1.695863	2.867348	19.212558
min	1.000000	40.000000	0.000000	4.000000	0.000000	10.000000
25%	3.000000	54.000000	0.000000	5.000000	2.000000	40.000000
50%	5.000000	69.000000	0.000000	7.000000	5.000000	55.000000
75%	7.000000	85.000000	1.000000	8.000000	7.000000	71.000000
max	9.000000	99.000000	1.000000	9.000000	9.000000	100.000000

```

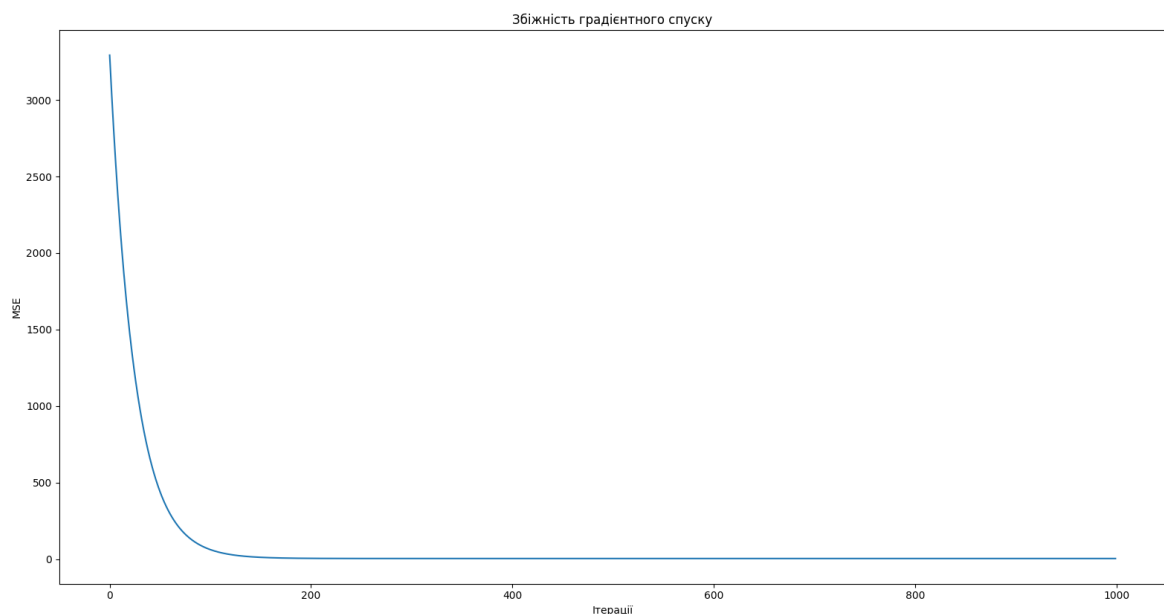
♦ Перевірка пропущених значень:
Hours Studied      0
Previous Scores    0
Extracurricular Activities  0
Sleep Hours        0
Sample Question Papers Practiced  0
Performance Index  0
dtype: int64

```

Цей графік показує силу та напрямок зв'язку між усіма змінними.

Висновок: між предикторами немає сильної мультиколінеарності (кореляція помірна).

Графік 2. Збіжність градієнтного спуску:



```

♦ Перевірка типів даних перед train/test:
Hours Studied          int64
Previous Scores        int64
Extracurricular Activities int64
Sleep Hours           int64
Sample Question Papers Practiced int64
Performance Index      float64
dtype: object

♦ Унікальні значення Extracurricular Activities:
[1 0]

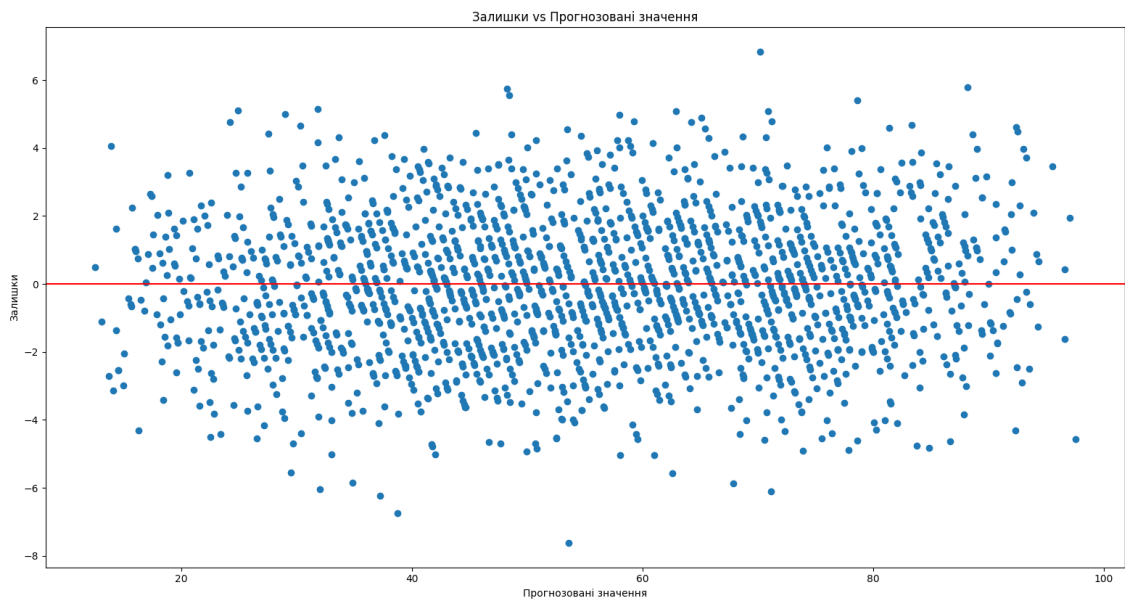
♦ Перевірка NaN після масштабування:
NaN y X_train: 0
NaN y X_test: 0

```

По осі X — ітерації, по осі Y — значення MSE.

Висновок: алгоритм швидко збігається (~до 200 ітерації), похибка виходить на плато → модель навчена стабільно.

Графік 3. Залишки vs Прогнозовані значення:



```

♦ Фінальні коефіцієнти:
[55.31149991  7.40134077 17.63727123  0.30429076  0.81003107  0.54884172]

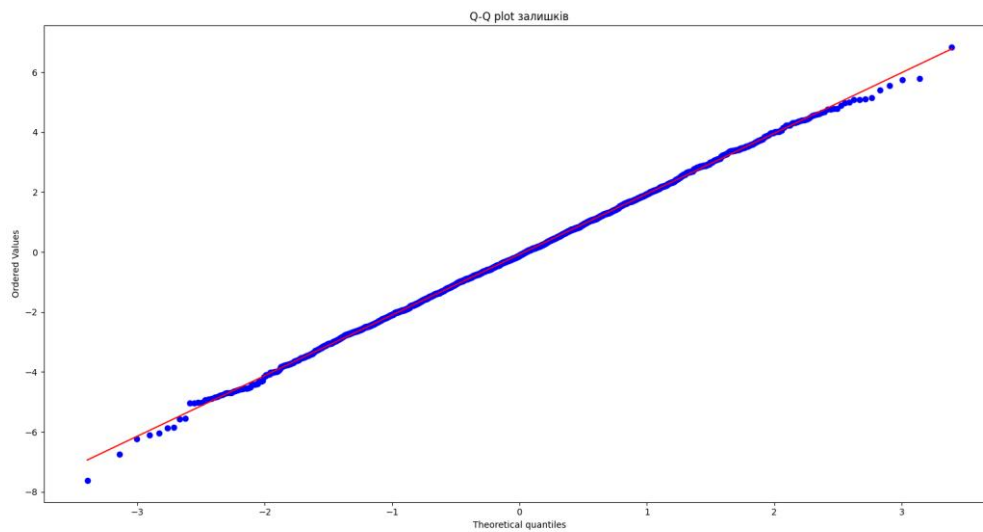
MSE (тест): 4.0826
R² (тест): 0.9890

```

По осі X — прогнозовані значення, по осі Y — залишки.

Висновок: залишки випадково розподілені навколо 0, отже, лінійність і гомоскедастичність виконуються.

Графік 4. Q-Q plot залишків:

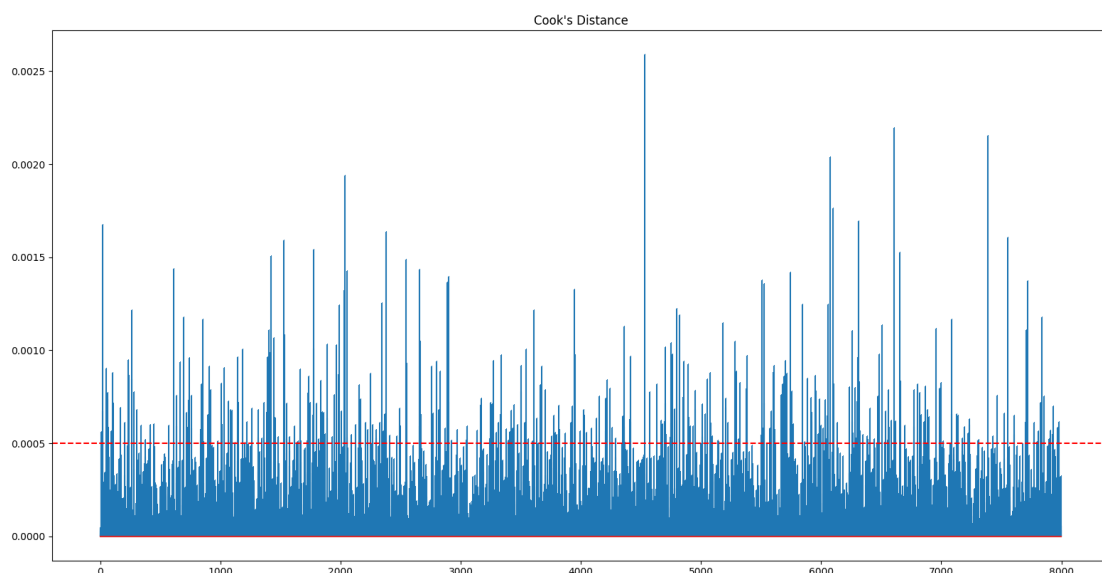


◆ VIF показники:		
	VIF	feature
0	1.000387	Hours Studied
1	1.000243	Previous Scores
2	1.000428	Extracurricular Activities
3	1.000461	Sleep Hours
4	1.000320	Sample Question Papers Practiced

Перевірка нормальності залишків.

Висновок: точки розташовані близько до діагоналі → залишки приблизно нормально розподілені.

Графік 5. Cook's Distance:




```
◆ Shapiro-Wilk test: ShapiroResult(statistic=np.float64(0.9996101861121233), pvalue=np.float64(0.9751496629506848))
◆ Durbin-Watson: 2.047191047494784
```

Визначає впливові точки (аномалії).

Висновок: Залишки моделі є нормально розподіленими і незалежними. Це підтверджує коректність побудованої регресійної моделі.

Висновок:

У ході виконання роботи, було реалізовано алгоритм множинної лінійної регресії методом градієнтного спуску з нуля та проведено повний аналіз побудованої моделі.

Було виконано попередню обробку даних: категоріальну змінну *Extracurricular Activities* закодовано у числовий формат, перевірено відсутність пропусків та некоректних значень, проведено масштабування ознак. У результаті сформовано якісний набір даних для моделювання.

Моделю навчено за допомогою градієнтного спуску, при цьому функція втрат (MSE) швидко збіглася до мінімуму, що свідчить про правильний підбір параметрів алгоритму. Отримані коефіцієнти регресії дозволяють інтерпретувати вплив кожної ознаки на підсумковий *Performance Index*. Якість моделі виявилась високою: $R^2 \approx 0.989$, що означає пояснення понад 98% варіації цільової змінної.

Також було перевірено 6 класичних припущень МНК:

- мультиколінеарність відсутня ($VIF \approx 1$),
- залишки нормально розподілені (Shapiro-Wilk, Q-Q plot),
- автокореляція відсутня (Durbin-Watson ≈ 2.04),
- лінійність та гомоскедастичність виконуються (графік «залишки vs прогнозовані значення»),
- впливових викидів практично немає (Cook's Distance).

Таким чином, побудована модель є коректною, статистично обґрунтованою і може бути використана для прогнозування академічної успішності студентів за вибраними ознаками.